

# Exempeldokument

Benjamin Kjellson

2015–12–18

## Gör detta först

Efter att ha öppnat denna fil (`exempel.Rmd`) i RStudio, tryck på knappen `Knit HTML` och läs vidare i det fönster som öppnas. Där ser du hur det som skrivs i `.Rmd`-filen kommer att se ut i slutändan.

## Detta dokument

Det här dokumentet visar exempel på några saker man kan göra i R Markdown. Tanken är att du tittar i filen `exempel.Rmd` för att se hur man skriver med R Markdown, och att du tittar på fönstret som poppar upp när du trycker på `Knit HTML` för att se hur resultatet ser ut.

## Några exempel

Notera att rubriken för detta stycke är mindre än den förra, och att de nedanför är ännu mindre.

En viktig sak vid rubriker är att ha en tom rad innan din rubrik. `##` Radbrytning för rubrik Så här ser det ut när raden ovanför inte är tom.

## Kursiv och fetstilt text

Här visas hur man får *kursiv text*; man kan också få detta *så här*. Här visas hur man får **fetstilt text**; man kan också få detta **så här**.

## Länkar

Vill du infoga en länk kan den klistras in som vanligt: [http://rmarkdown.rstudio.com/authoring\\_basics.html](http://rmarkdown.rstudio.com/authoring_basics.html) eller göras till en hyperlänk.

## Listor

På detta vis kan man göra en lista:

- Första
- Andra
  - Tredje
  - Fjärde

Notera att listan ovan- och nedanför omges av en blank rad, samt att underlistans element börjar med fyra mellanslag innan +-tecknet som startar listan. Vi kan göra en numrerad lista också:

1. Item 1
2. Item 2
3. Item 3
  - Item 3a
  - Item 3b

## Rad- och sidbyten

Vill vi börja på ett nytt stycke lämnar vi en tom rad mellan det nya stycket och det förra. Vill vi börja på en ny rad utan att börja ett nytt stycke så sätter vi två mellanslag och börjar skriva på nästa rad. **Notera** att ett eller noll mellanslag och ny rad inte påverkar utseendet, så tveka inte på att börja på en ny rad även mitt i en mening. Det ökar läsbarheten i .Rmd-filen.

Sidbyten (eller långa horisontella streck) kan göras så här:

\_\_\_\_\_

eller så här:

\_\_\_\_\_

## Tabeller

Följande är ett exempel på en tabell som skapats manuellt (inte i R):

| Frukt  | Antal |
|--------|-------|
| Äpplen | 5     |
| Päron  | 10    |

## Kod

Vi kan även skriva kod i R och använda den i dokumentet. Vill jag t.ex. räkna ut vad  $34/7$  är så kan jag göra det; det borde bli 4.8571429. Men ofta vill jag inte ha så många decimaler; två kanske räcker i detta fall. Det får jag enkelt genom att använda mig av R-funktionen `round`. Avrundat blir svaret alltså 4.86.

Vill jag ha större stycken kod som visas så kan jag göra det så här:

```
x <- 5      # Definiera en variabel x
y <- x + 5   # Det som skrivs på samma rad efter # är kommentarer, förresten
x <- 0      # Jag ändrar värdet på x. Men vad är nu värdet på y?

# Värdet på y kommer att visas om jag gör så här:
y
```

```
## [1] 10
```

De två # är en så kallad kommentar och är mest till för att göra det lättare att skilja mellan koden och utskrifter. Om man vill välja en egen kommentar kan man göra detta så här:

```
y
```

```
OUTPUT [1] 10
```

Om man tycker det är fult och vill ta bort kommentarerna kan man göra detta så här:

```
y
```

```
[1] 10
```

Jag kan även skriva kod som inte visas i det slutgiltiga dokumentet:

Eller så kan jag skriva kod som syns, men som inte utvärderas:

```
1 + 2 + 3 + 4
```

**Plottar** Om jag vill visa en graf kan jag göra detta t.ex. så här:

```
plot(cars)
```

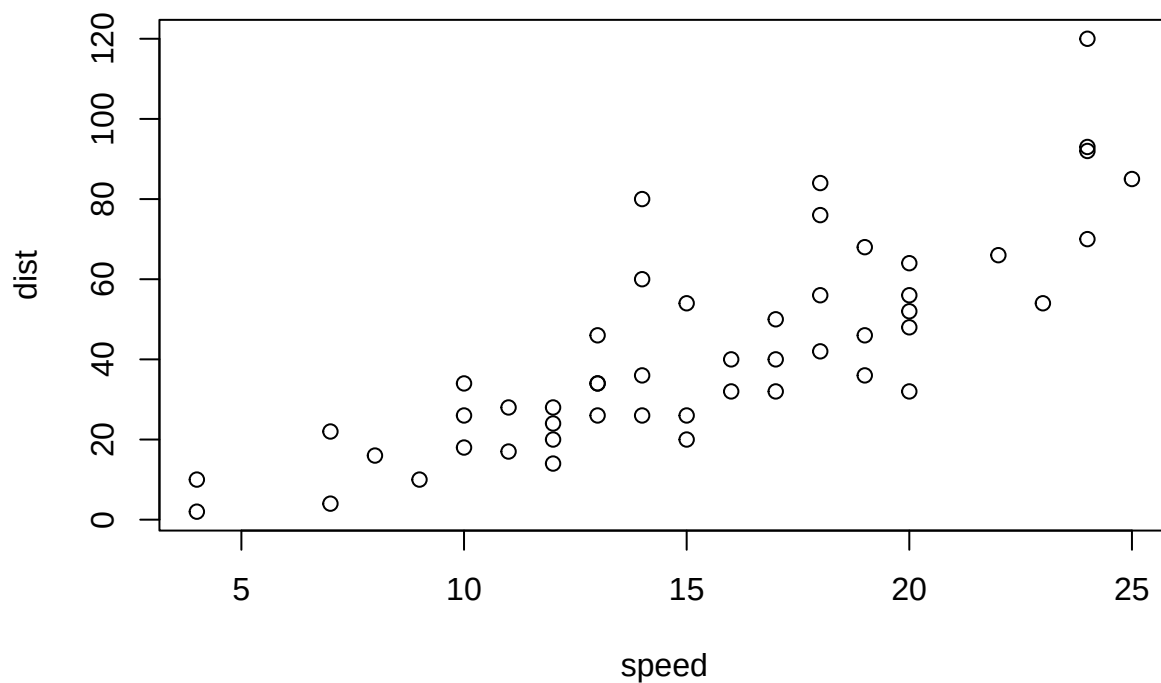


Figure 1: Ett spridningsdiagram.

Notera att du inte behöver skriva `fig.cap='Diagram 1: Ett spridningsdiagram.'`, då detta numreras automatiskt när du kör `knit to pdf` (men numreringen dyker inte upp när du kör `knit to html`). Varje

plot du tar med i en rapport bör ha ett nummer, så att det inte blir några missförstånd om vilket diagram du refererar till i texten. Du kommer kanske skriva något i stil med “Vi ser sedan i Diagram 1 att ...”.

Ovan använde jag mig av den inbyggda datamängden `cars`. Det finns ett flertal inbyggda datamängder i R som man kan använda sig av om man vill jobba med exempel, så som `cars`, `pressure` och `iris`. För att se vilka data set som finns tillgängliga så kan du skriva in `data()` i konsollen.

Vill jag bara se grafen och ingen kod så går det också genom att skriva med `echo=false` i kodblocket:

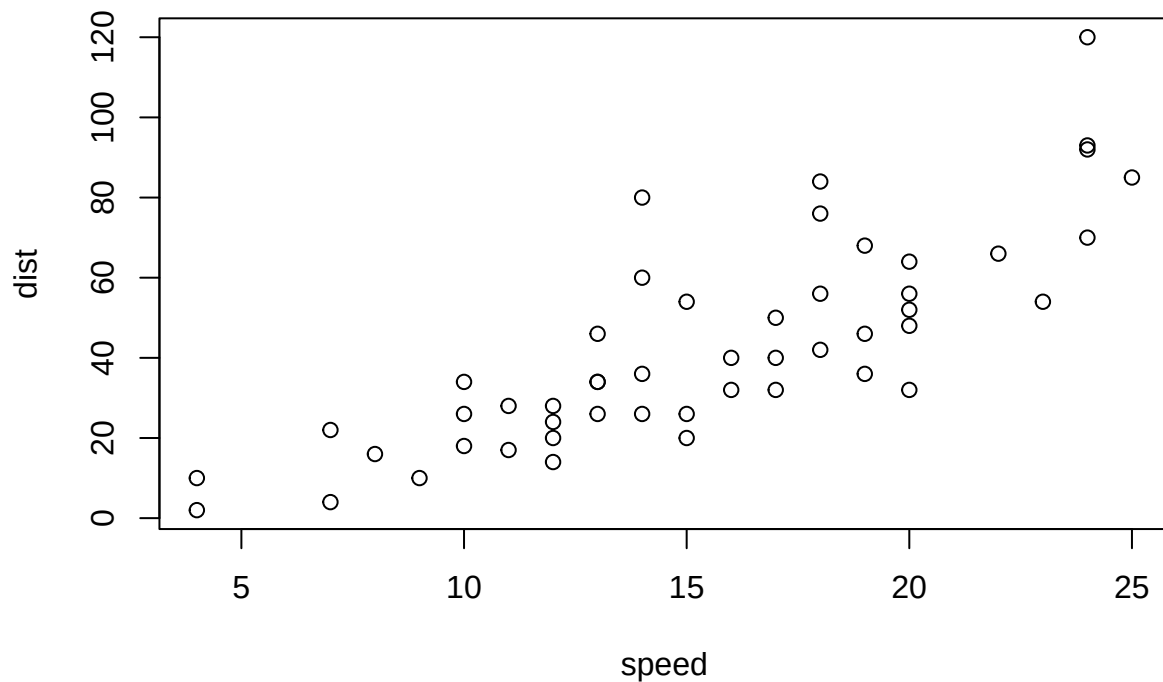


Figure 2: Samma spridningsdiagram.

Detta gör helt enkelt så att koden inte visas, men fortfarande körs, och används framförallt när man ska visa plottar.

Det finns en del saker att tänka på när man skapar plottar med beskrivande texter. Det första är att alltid ha en tom rad efter kodblocket. Annars så kommer den beskrivande texten ej att dyka upp. Det andra viktiga att tänka på är att om du vill skapa flera plottar och samtidigt använda `fig.cap` för att lägga till en beskrivande text, kan du endast ha en plot per kodblock, annars kommer plottarna ha samma beskrivande text vilket man oftast vill undvika eftersom man då inte kan beskriva plottarna var för sig. Gör **ej** som följande:

```
plot(cars)
```

```
plot(pressure)
```

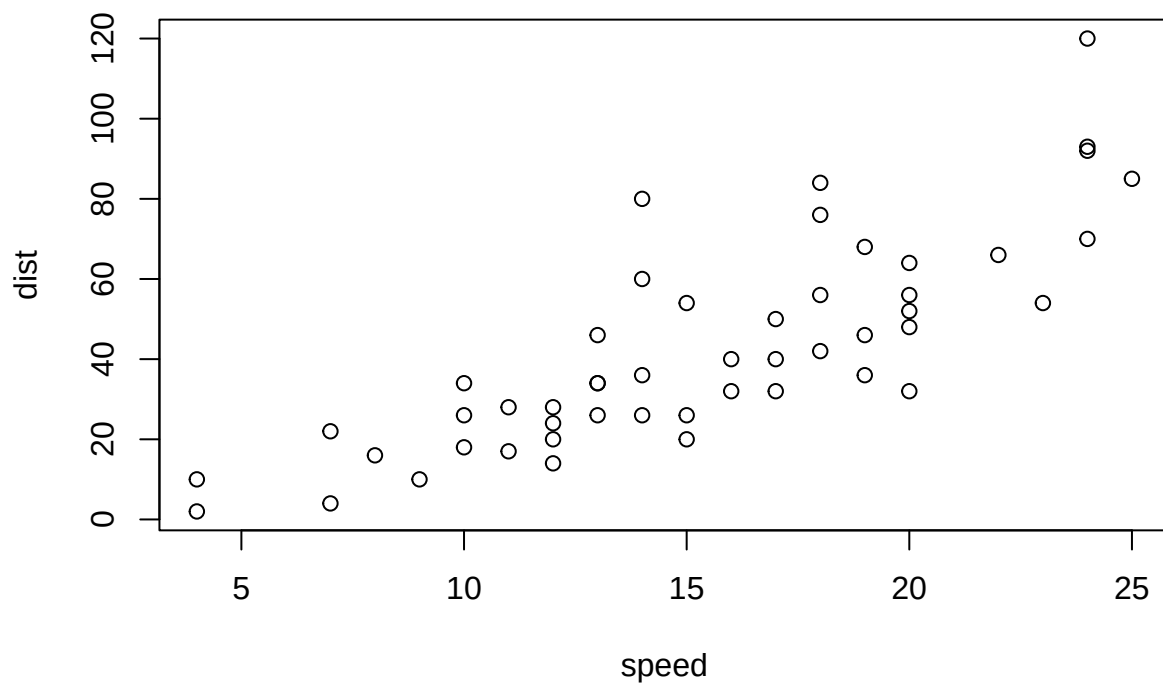


Figure 3: Ett kodblock med två plottar.

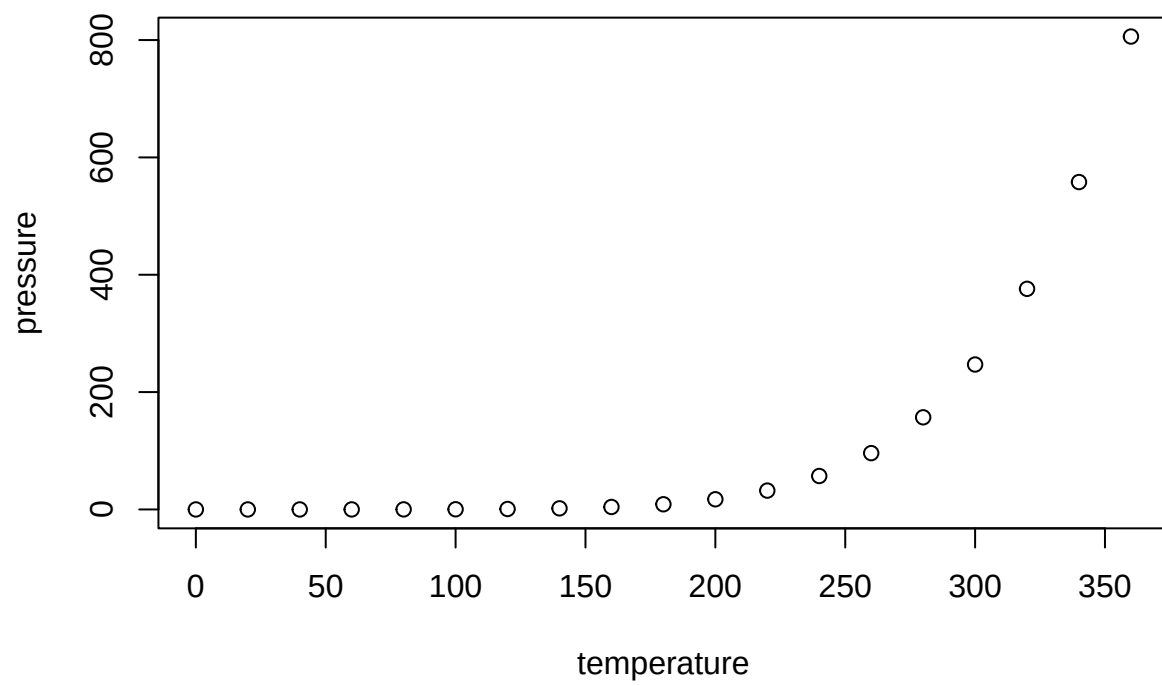


Figure 4: Ett kodblock med två plottar.

**Tabeller** Kanske vill jag ha med en tabell som jag inte behöver skapa manuellt som ovan. Vi skapar en enkel data frame i R först:

```
min_tabell <- data.frame(Äpplen = c(3, 4),
                        Päron = c(9, 5),
                        Apelsiner = c(10, 20),
                        Bananer = c(5, 2),
                        Adress = c("Villovägen 13", "Återvändsgränd 5"))
# Vi lägger till namn i raden längst till vänster i tabellen:
rownames(min_tabell) <- c("Butik A", "Butik B")
```

Sedan visar vi tabellen:

```
min_tabell # Gör att tabellen visas nedanför
```

|         | Äpplen | Päron | Apelsiner | Bananer | Adress           |
|---------|--------|-------|-----------|---------|------------------|
| Butik A | 3      | 9     | 10        | 5       | Villovägen 13    |
| Butik B | 4      | 5     | 20        | 2       | Återvändsgränd 5 |

Men vi kan få den att se snyggare ut så här:

```
knitr::kable(min_tabell, caption = "Tabell 1: Snyggare tabell med text.")
```

Table 1: Tabell 1: Snyggare tabell med text.

|         | Äpplen | Päron | Apelsiner | Bananer | Adress           |
|---------|--------|-------|-----------|---------|------------------|
| Butik A | 3      | 9     | 10        | 5       | Villovägen 13    |
| Butik B | 4      | 5     | 20        | 2       | Återvändsgränd 5 |

Notera att texten till tabellen ges till funktionen `knitr::kable`, inte genom `fig.cap` som vi använde oss av när vi gjorde en plot.

**Att generera slumpstal** I laboration 2 och 3 kommer du att generera slumpstal från olika fördelningar. Eftersom de är slumpstal kan man tänka sig att samma stycke kod kan producera olika resultat varje gång du kör den eftersom slump är inblandat. Ett problem som då kan uppstå är att du kanske försöker beskriva det som du ser i en plot första gången du kör koden, men när du kör om koden så har utseendet på plotten förändrats drastiskt p.g.a. slumpen.

Det finns ett sätt att undvika detta problem. Som du kommer få veta inför laboration 2 så är vanliga slumpstalsgeneratorer egentligen helt deterministiska. Faktum är att det i R går att ge ett "frö" —ett sorts startvärde— till slumpstalsgeneratorn, som gör att all slumpstalsgenerering som sker efteråt går att återskapa exakt. Nedan visas vad som menas:

```
# Utan att sätta ett frövärde: skillnad
slump1 <- runif(4) # generera 4 likformigt fördelade slumpstal
slump2 <- runif(4) # generera 4 ANDRA likformigt fördelade slumpstal

# Sätter ett frövärde: ingen skillnad
set.seed(19880210) # Ger mitt födelsedatum som ett "frö".
ingen_slump1 <- runif(4) # generera 4 likformigt fördelade slumpstal
```

```
ingen_slump2 <- runif(4) # generera 4 likformigt fördelade slumpstal
```

```
set.seed(19880210) # Sätter samma frövärde på nytt
```

```
ingen_slump3 <- runif(4) # generera 4 likformigt fördelade slumpstal
```

```
ingen_slump4 <- runif(4) # generera 4 likformigt fördelade slumpstal
```

```
slump1
```

```
## [1] 0.4047071 0.7985359 0.8223167 0.2147888
```

```
slump2
```

```
## [1] 0.2185903 0.3433926 0.8319796 0.2044843
```

```
ingen_slump1
```

```
## [1] 0.3152433 0.6465920 0.1349646 0.5479665
```

```
ingen_slump2
```

```
## [1] 0.02966373 0.78734708 0.95372176 0.17156849
```

```
ingen_slump3
```

```
## [1] 0.3152433 0.6465920 0.1349646 0.5479665
```

```
ingen_slump4
```

```
## [1] 0.02966373 0.78734708 0.95372176 0.17156849
```

I laboration 2 och 3 kommer du att se hur du ska använda funktionen `set.seed` med ett frö, på ett sådant sätt att dina labbrapporter går att återskapa till minsta detalj för någon som har tillgång till din kod.